



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

IT0049 (WEB SYSTEM TECHNOLOGIES)

TECHNICAL FORMATIVE ASSESSMENT

2

FROM ARRAYS TO A REAL DATABASE

Student Name / Group Name:		
Members (if Group):	Name	Role
Section:		
Professor:		

PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design, implement and evaluate computer-based systems or applications to meet desired needs and requirements. [PO: C]

COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- Apply advanced web development principles in creating, debugging, validating, and securing database-backed web applications. [CLO: 2]

INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Configure CodeIgniter's database connection settings.
- Create a MySQL database and tables using a provided schema.
- Build CodeIgniter Models for the Customer Accounts and User Accounts pages.
- Retrieve records with Query Builder and display them in a view.

BACKGROUND INFORMATION

The POS application from the previous activity stored its data in static PHP arrays, which reset every time the server restarts. This activity replaces that temporary storage with a real MySQL database. CodeIgniter's Model class wraps a database table and gives you Query Builder methods — such as `findAll()` — for retrieving records without writing raw SQL.

A good rule for this activity: once a page's data comes from a Model, its controller and view code should look almost identical to the static-array version. Only where the data comes from changes.

GRADING SYSTEM / RUBRIC

CRITERIA	Description
Functionality & Requirements Completeness (40 points max)	All required pages and features are present and work as specified in the activity instructions.
Code Structure & Organization (25 points max)	Controllers, models, and views are separated correctly and follow the framework's conventions; code is readable and sensibly named.
Database Design & Query Usage (20 points max)	The database and tables match the provided schema, records are retrieved through a Model using Query

CRITERIA	Description
	Builder (not raw SQL), and the results display correctly in the view.
Documentation & Submission Quality (15 points max)	The GitHub repository is organized and complete, the README explains how to set up and run the project, and the hosted link is live and matches the submitted code.

LABORATORY ACTIVITY

You will extend the POS application from TFA1 by replacing its static arrays with a real MySQL database. Use the schema below.

1. Configure your CodeIgniter .env database settings to connect to a local MySQL database.
2. Create the database and the customers and users tables using the schema below:

```
CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL,
  phone VARCHAR(20),
  created_at DATETIME NOT NULL
);

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  full_name VARCHAR(100) NOT NULL,
  created_at DATETIME NOT NULL
);
```

3. Populate each table with at least 5 sample records.
4. Create a CustomerModel and a UserModel.
5. Update your Customer Accounts and User Accounts controllers to retrieve records through their Models instead of static arrays.
6. Confirm the views display the retrieved database records the same way they displayed the static array data in TFA1.

Submission Requirements:

- A link to a GitHub repository containing your raw project files, including your database export.
- A link to your hosted, working version of the application.

REFERENCES

- Manfield, A. (2021). Pro CodeIgniter: Learn how to create professional web-applications with PHP.
- CodeIgniter User Guide. Models.
<https://codeigniter4.github.io/userguide/models/model.html>
- CodeIgniter User Guide. Query Builder.
https://codeigniter4.github.io/userguide/database/query_builder.html

Database Integration Implementation

IT0049 Technical Formative Assessment 2

This implementation continues the existing SimplePOS CodeIgniter 4 website. The Customer Accounts and User Accounts pages now retrieve persistent records from the local MySQL database through CodeIgniter Models and Query Builder instead of static PHP arrays.

Implementation Summary

- Connected CodeIgniter to the existing pos_database database in XAMPP.
- Created CustomerModel and UserModel for the customer_accounts and user_accounts tables.
- Replaced the static arrays in both controllers with Model queries.
- Kept the existing routes and page design while adding database-backed status fields.
- Exported the working database as database/pos_database.sql for repository submission.

Database Schema Used

The implementation uses the corrected account schema prepared for the preceding AI-reviewed database activity. These table names and fields are more detailed than the simplified example in the activity sheet, while still supporting the same Customer Accounts and User Accounts views.

Customer Accounts Table

Field	Data Type	Constraint or Purpose
customer_id	INT UNSIGNED	Primary key and auto increment
first_name	VARCHAR(50)	Required customer first name
last_name	VARCHAR(50)	Required customer last name
email	VARCHAR(100)	Required and unique
phone	VARCHAR(20)	Required and preserves leading zero
address	VARCHAR(255)	Optional address
account_status	VARCHAR(20)	Required with Active default
created_at	DATETIME	Required with current timestamp default

User Accounts Table

Field	Data Type	Constraint or Purpose
user_id	INT UNSIGNED	Primary key and auto increment
username	VARCHAR(50)	Required and unique
password_hash	VARCHAR(255)	Required password hash storage
first_name	VARCHAR(50)	Required staff first name

Field	Data Type	Constraint or Purpose
last_name	VARCHAR(50)	Required staff last name
email	VARCHAR(100)	Required and unique
role	VARCHAR(20)	Required staff role
account_status	VARCHAR(20)	Required with Active default
created_at	DATETIME	Required with current timestamp default

Sample Database Records

Five Customer Records

ID	Customer	Email	Phone	Status
1	Juan Dela Cruz	juan@example.com	09171234567	Active
2	Maria Santos	maria@example.com	09181234567	Active
3	Carlo Reyes	carlo@example.com	09191234567	Active
4	Ana Garcia	ana@example.com	09201234567	Inactive
5	Luis Mendoza	luis@example.com	09211234567	Active

Five User and Staff Records

ID	Username	Staff Name	Role	Status
1	admin01	John Admin	Admin	Active
2	cashier01	Ella Cruz	Cashier	Active
3	cashier02	Mark Tan	Cashier	Active
4	manager01	Sofia Lim	Manager	Active
5	cashier03	Paul Ramos	Cashier	Inactive

XAMPP and SQL Requirement

No SQL needs to be pasted into phpMyAdmin on the current computer because pos_database, customer_accounts, user_accounts, and all ten sample records already exist and were successfully queried.

For a different computer, repository reviewer, or clean XAMPP installation, start Apache and MySQL, open phpMyAdmin, choose Import, and select database/pos_database.sql. Importing the file creates the database, both tables, their keys and constraints, and the sample records automatically.

CodeIgniter Database Connection

The project .env file contains the local XAMPP connection settings below.

```
database.default.hostname = 127.0.0.1
database.default.database = pos_database
database.default.username = root
database.default.password =
database.default.DBDriver = MySQLi
database.default.DBPrefix =
database.default.port = 3306
```

CodeIgniter Models

Each Model identifies its MySQL table, primary key, return type, and allowed fields.

CustomerModel

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class CustomerModel extends Model
{
    protected $table = 'customer_accounts';
    protected $primaryKey = 'customer_id';
    protected $returnType = 'array';
    protected $useAutoIncrement = true;
    protected $allowedFields = [
        'first_name', 'last_name', 'email', 'phone',
        'address', 'account_status',
    ];
}
```

UserModel

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class UserModel extends Model
{
    protected $table = 'user_accounts';
    protected $primaryKey = 'user_id';
    protected $returnType = 'array';
    protected $useAutoIncrement = true;
    protected $allowedFields = [
        'username', 'password_hash', 'first_name', 'last_name',
        'email', 'role', 'account_status',
    ];
}
```

Controller Queries

The controllers use Model methods backed by Query Builder. CONCAT creates the full_name value expected by the existing views, so the TFA1 page structure remains familiar.

Customers Controller Query

```
$customerModel = new CustomerModel();
$customers = $customerModel
    ->select("CONCAT(first_name, ' ', last_name) AS full_name", false)
    ->select('email, phone, account_status')
    ->orderBy('customer_id', 'ASC')
    ->findAll();
```


Users Controller Query

```
$userModel = new UserModel();  
$users = $userModel  
    ->select('username')  
    ->select("CONCAT(first_name, ' ', last_name) AS full_name", false)  
    ->select('role, account_status')  
    ->orderBy('user_id', 'ASC')  
    ->findAll();
```

View Integration

The existing foreach loops are retained. The views now receive Model results and escape every displayed value with `esc()`. The customer page shows name, email, phone, and status. The user page shows username, name, role, and status. The `password_hash` field is never selected or displayed.

Customer View Loop

```
<?php foreach ($customers as $customer): ?>
    <tr>
        <td><?= esc($customer['full_name']) ?></td>
        <td><?= esc($customer['email']) ?></td>
        <td><?= esc($customer['phone']) ?></td>
        <td><?= esc($customer['account_status']) ?></td>
    </tr>
<?php endforeach ?>
```

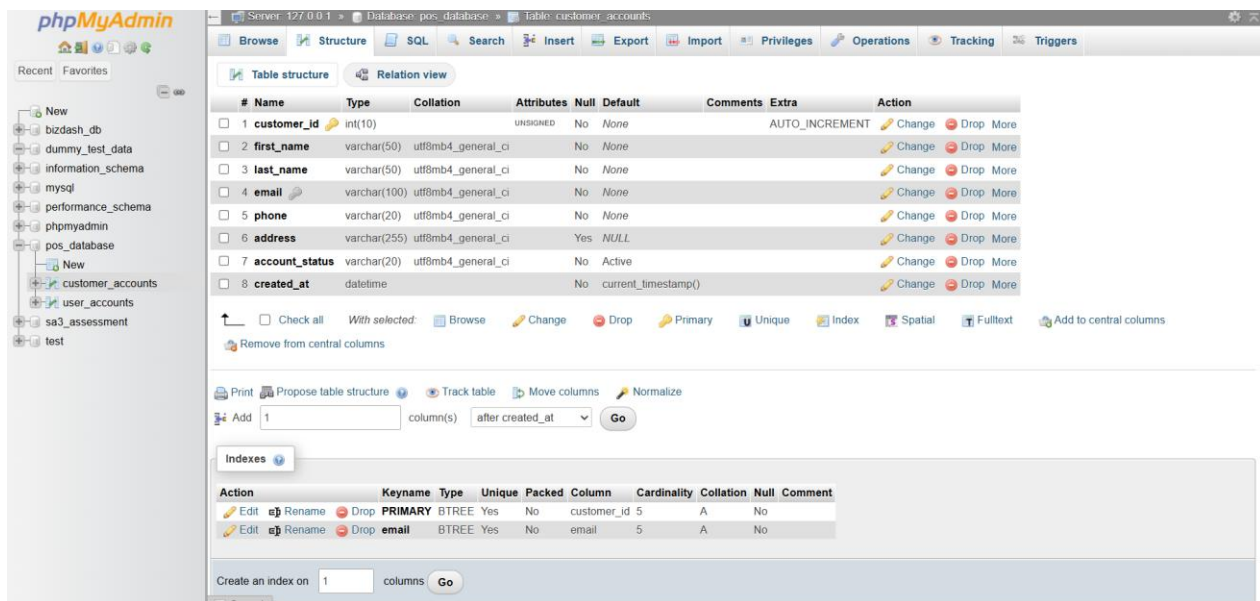
User View Loop

```
<?php foreach ($users as $user): ?>
    <tr>
        <td><?= esc($user['username']) ?></td>
        <td><?= esc($user['full_name']) ?></td>
        <td><?= esc($user['role']) ?></td>
        <td><?= esc($user['account_status']) ?></td>
    </tr>
<?php endforeach ?>
```

Verification Results

Check	Result
Customer page HTTP response	200 OK
Customer rows displayed	5
Known customer found	Juan Dela Cruz
User page HTTP response	200 OK
User rows displayed	5
Known username found	admin01
PHP syntax checks	Passed for both Models and controllers

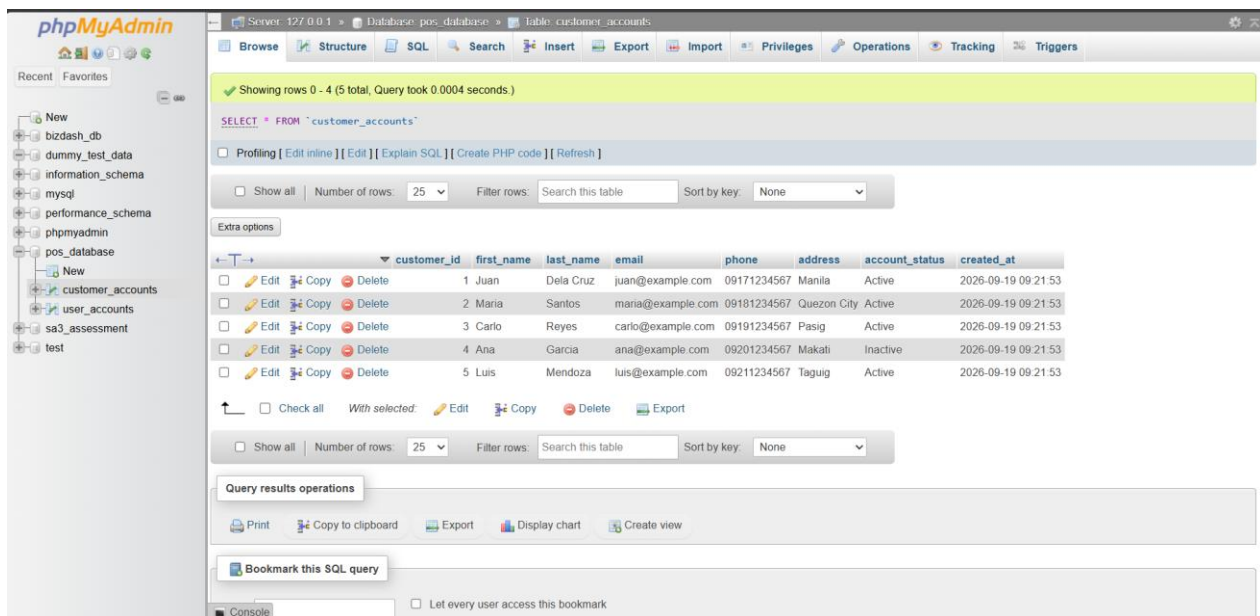
Database Evidence



The screenshot shows the phpMyAdmin interface with the 'customer_accounts' table structure displayed. The table has 8 columns: customer_id (int(10), UNSIGNED, No, None, AUTO_INCREMENT), first_name (varchar(50), utf8mb4_general_ci, No, None), last_name (varchar(50), utf8mb4_general_ci, No, None), email (varchar(100), utf8mb4_general_ci, No, None), phone (varchar(20), utf8mb4_general_ci, No, None), address (varchar(255), utf8mb4_general_ci, Yes, NULL), account_status (varchar(20), utf8mb4_general_ci, No, Active), and created_at (datetime, No, current_timestamp()). The 'customer_id' column is the primary key. There is a unique index on the 'email' column.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	customer_id	int(10)	utf8mb4_general_ci	UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	first_name	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
3	last_name	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
4	email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
5	phone	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
6	address	varchar(255)	utf8mb4_general_ci		Yes	NULL			Change Drop More
7	account_status	varchar(20)	utf8mb4_general_ci		No	Active			Change Drop More
8	created_at	datetime	utf8mb4_general_ci		No	current_timestamp()			Change Drop More

Figure 1 customer_accounts field types, primary key, auto increment, and unique email index



The screenshot shows the phpMyAdmin interface displaying five customer account records. The records are as follows:

customer_id	first_name	last_name	email	phone	address	account_status	created_at
1	Juan	Dela Cruz	juan@example.com	09171234567	Manila	Active	2026-09-19 09:21:53
2	Maria	Santos	maria@example.com	09181234567	Quezon City	Active	2026-09-19 09:21:53
3	Carlo	Reyes	carlo@example.com	09191234567	Pasig	Active	2026-09-19 09:21:53
4	Ana	Garcia	ana@example.com	09201234567	Makati	Inactive	2026-09-19 09:21:53
5	Luis	Mendoza	luis@example.com	09211234567	Taguig	Active	2026-09-19 09:21:53

Figure 2 Five customer account records in phpMyAdmin

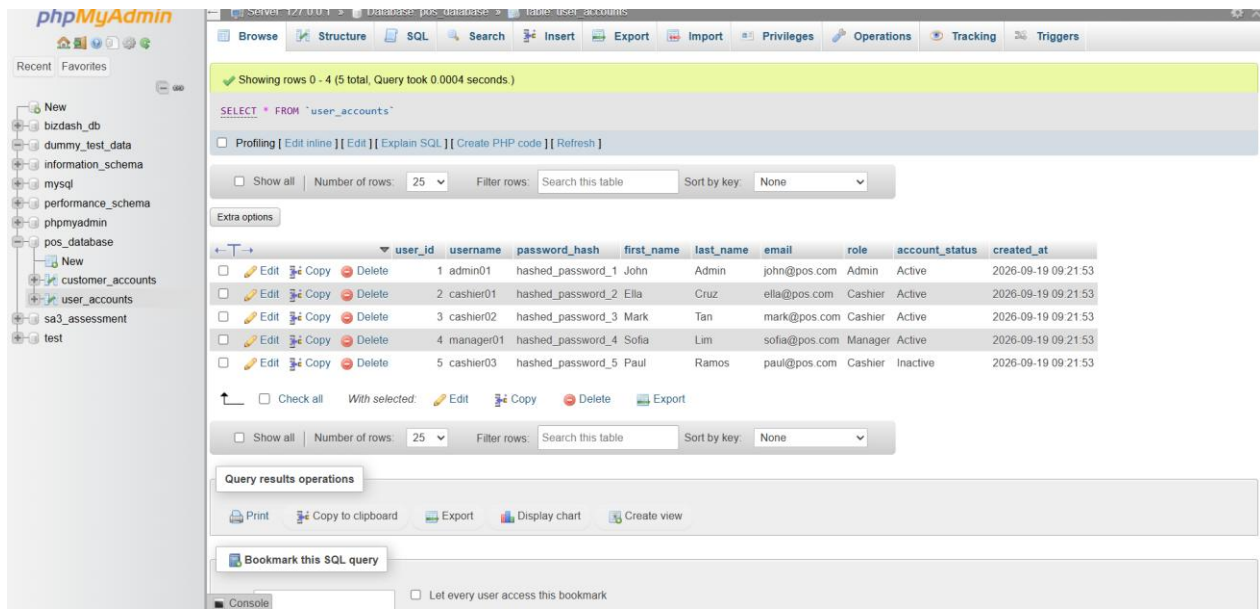


Figure 3 Five user and staff account records in phpMyAdmin

Short Reflection

Replacing the static arrays with Models made the account pages persistent without changing the overall MVC flow. The controllers now request records from MySQL, while the views continue to loop through arrays returned by CodeIgniter. This activity also showed why field names, primary keys, unique constraints, and safe output escaping must agree across the database, Model, controller, and view.

Final Submission Checklist

Submission Item	Status
CodeIgniter project files	Completed in the TA1 project folder
Database export	Completed as database/pos_database.sql
Database-backed customer page	Completed and verified
Database-backed user page	Completed and verified
GitHub repository link	Add after uploading the project
Hosted application link	Add after deploying the project